

RestN

Generated by Doxygen 1.16.1

1 Пользовательская документация (RestN)	1
1.1 Обзор проекта	1
1.2 Технологии программирования	1
1.3 Структура проекта	2
1.4 Описание программных модулей	2
1.4.1 Модуль работы с базой данных (DBWorker)	2
1.4.2 Модуль сетевого клиента (TcpClient)	3
1.4.3 Модуль сетевого сервера (TcpServer + ClientConnection)	3
1.4.4 Модуль логирования (Logs)	3
1.4.5 Модуль настроек (Settings)	3
1.4.6 Пользовательский интерфейс (QML)	3
1.5 Интеграция программных модулей	3
1.5.1 Подключаемые внешние модули	3
1.5.2 Интеграция C++ и QML	4
1.5.3 Клиент-серверное взаимодействие	4
1.6 Руководство пользователя	4
1.6.1 Начальный запуск	4
1.6.2 Окно авторизации	4
1.6.3 Диалог настроек	4
1.6.4 Главное окно и выдвижное меню	4
1.6.5 Оформление заказа (страница для официанта)	5
1.6.6 Управление складом (страница для кладовщика)	5
1.6.7 Отчёты (страница для администратора)	5
1.7 Настройки и системные файлы	5
1.8 Сборка проекта (для разработчиков)	6
1.9 Контакты	6
2 Hierarchical Index	7
2.1 Class Hierarchy	7
3 Class Index	9
3.1 Class List	9
4 File Index	11
4.1 File List	11
5 Class Documentation	13
5.1 ClientConnection Class Reference	13
5.1.1 Detailed Description	13
5.1.2 Constructor & Destructor Documentation	13
5.1.2.1 ClientConnection()	13
5.2 DatabaseEx Class Reference	14
5.2.1 Detailed Description	14

5.3 DBWorker Class Reference	14
5.3.1 Detailed Description	16
5.3.2 Constructor & Destructor Documentation	16
5.3.2.1 DBWorker()	16
5.3.3 Member Function Documentation	16
5.3.3.1 addOrderItem()	16
5.3.3.2 addProductQuantity()	16
5.3.3.3 checkAndConsume()	17
5.3.3.4 checkUser()	17
5.3.3.5 createOrder()	17
5.3.3.6 getFoodPopularity()	18
5.3.3.7 getLowStockWarning()	18
5.3.3.8 getReportRevenue()	18
5.3.3.9 getRevenueByDay()	19
5.3.3.10 pingServer()	19
5.3.3.11 reconnect()	19
5.3.3.12 returnIngredients()	20
5.3.3.13 saveReportToCSV()	20
5.3.3.14 updateProductQuantity()	21
5.4 Exceptions Class Reference	21
5.4.1 Detailed Description	21
5.4.2 Constructor & Destructor Documentation	22
5.4.2.1 Exceptions()	22
5.5 FileEx Class Reference	22
5.5.1 Detailed Description	22
5.6 Logs Class Reference	22
5.6.1 Detailed Description	23
5.6.2 Member Function Documentation	23
5.6.2.1 setLogPath()	23
5.7 NetworkEx Class Reference	23
5.7.1 Detailed Description	23
5.8 Settings Class Reference	24
5.8.1 Detailed Description	25
5.8.2 Constructor & Destructor Documentation	25
5.8.2.1 Settings()	25
5.9 TcpClient Class Reference	25
5.9.1 Detailed Description	27
5.9.2 Constructor & Destructor Documentation	27
5.9.2.1 TcpClient()	27
5.9.3 Member Function Documentation	27
5.9.3.1 addOrderItem()	27
5.9.3.2 addProductQuantity()	28

5.9.3.3 checkAndConsume()	28
5.9.3.4 checkUser()	28
5.9.3.5 connectToServer()	29
5.9.3.6 createOrder()	29
5.9.3.7 getFoodPopularity()	29
5.9.3.8 getFoods()	30
5.9.3.9 getLowStockWarning()	30
5.9.3.10 getProducts()	30
5.9.3.11 getReportRevenue()	30
5.9.3.12 getRevenueByDay()	31
5.9.3.13 getUserId()	31
5.9.3.14 getUsername()	31
5.9.3.15 getUserRole()	31
5.9.3.16 isConnected()	31
5.9.3.17 pingServer()	32
5.9.3.18 reconnect()	32
5.9.3.19 returnIngredients()	32
5.9.3.20 saveReportToCSV()	32
5.9.3.21 updateProductQuantity()	33
5.10 TcpServer Class Reference	33
5.10.1 Detailed Description	34
5.10.2 Constructor & Destructor Documentation	34
5.10.2.1 TcpServer()	34
5.10.3 Member Function Documentation	34
5.10.3.1 start()	34
5.11 ValidateEx Class Reference	34
5.11.1 Detailed Description	35
6 File Documentation	37
6.1 dbworker.h	37
6.2 exceptions.h	38
6.3 logs.h	38
6.4 settings.h	39
6.5 tcpclient.h	40
6.6 tcpserver.h	41

Глава 1

Пользовательская документация (RestN)

1.1. Обзор проекта

Название: RestN

Тип: кроссплатформенное приложение для планшетов (Android / Windows / Linux) на Qt 6

Назначение: автоматизация приёма заказов, управления складом и формирования отчётности в ресторане.

Ключевые возможности:

- Авторизация сотрудников (официант, кладовщик, администратор)
 - Оформление заказов с автоматическим списанием ингредиентов
 - Возврат продуктов при удалении позиции из заказа
 - Управление складскими остатками (добавление, убавка, установка)
 - Отчёты по выручке, популярности блюд и дефицитным продуктам
 - Сохранение отчётов в CSV
 - Ролевое разграничение доступа
 - Синхронизация данных по локальной сети (клиент-сервер)
-

1.2. Технологии программирования

При разработке использовались следующие инструменты и подходы:

- **Язык C++** – основная логика приложения, работа с сетью и базой данных.
- **Qt 6** – фреймворк для графического интерфейса (Qt Quick), сетевых сокетов и доступа к SQL.
- **PostgreSQL** – централизованная СУБД для серверного режима.
- **CMake** – система сборки проекта.
- **Qt Creator** – интегрированная среда разработки.

Архитектурные подходы:

Клиент-серверная архитектура, паттерн Model-View (данные передаются через QVariantList), механизм сигналов и слотов для коммуникации между объектами, а также Singleton для классов [Logs](#) и [Settings](#).

1.3. Структура проекта

```

project/
├── CMakeLists.txt
├── include/           # заголовочные файлы C++
│   ├── dbworker.h
│   ├── logs.h
│   ├── tcpclient.h
│   ├── tcpserver.h
│   ├── settings.h
│   └── exceptions.h
├── src/              # исходные файлы C++
│   ├── dbworker.cpp
│   ├── logs.cpp
│   ├── tcpclient.cpp
│   ├── tcpserver.cpp
│   ├── settings.cpp
│   ├── exceptions.cpp
│   └── main.cpp
├── resources/        # ресурсы (иконки, стили)
│   ├── qml/          # файлы интерфейса QML
│   │   ├── Main.qml
│   │   ├── LoginPage.qml
│   │   ├── OrderPage.qml
│   │   ├── StockPage.qml
│   │   └── ReportsPage.qml
│   └── icons/
├── tests/            # модульные тесты
└── build/            # каталог сборки (игнорируется)

```

1.4. Описание программных модулей

1.4.1. Модуль работы с базой данных (DBWorker)

Класс `DBWorker` предназначен для выполнения всех операций с базой данных PostgreSQL. Открывает соединение в конструкторе, поддерживает переподключение через `reconnect`. Основные методы:

- `checkUser(login, password)` – аутентификация с хешированием SHA3-256 + соль.
- `getFoods()` – возвращает список блюд с доступностью (рассчитывается по остаткам) и составом.
- `getProducts()` – список продуктов с остатками и минимальным порогом.
- `updateProductQuantity()` / `addProductQuantity()` – изменение остатков.
- `checkAndConsume(foodId, quantity)` – в транзакции проверяет наличие ингредиентов и списывает их.
- `returnIngredients(foodId, quantity)` – возвращает ингредиенты на склад.
- `createOrder()` / `addOrderItem()` – создание заказа.
- Методы отчётов: `getReportRevenue`, `getRevenueByDay`, `getFoodPopularity`, `getReportLowStock`.
- `saveReportToCSV()` – сохранение отчёта в CSV.

Все публичные методы обрабатывают исключения (`DatabaseEx`, `ValidateEx`), логируют ошибки и испускают сигнал `errorOccurred` для QML.

1.4.2. Модуль сетевого клиента (TcpClient)

Класс `TcpClient` дублирует интерфейс `DBWorker`, но вместо прямых запросов к БД отправляет JSON-команды на сервер. Соединение устанавливается методом `connectToServer(host, port)`. Протокол обмена: запрос/ответ в виде строк JSON, завершённых символом `\n`. Для синхронного ожидания используется `waitForResponse` с таймаутом 60 секунд. При потере соединения генерируется сигнал `errorOccurred`.

Пример запроса:

```
{"command": "checkUser", "requestId": 1, "data": {"login": "waiter1", "password": "123"}}
```

Ответ сервера:

```
{"response": true, "requestId": 1, "data": {"role": "waiter", "name": "Иванов", "userId": 5}}
```

1.4.3. Модуль сетевого сервера (TcpServer + ClientConnection)

`TcpServer` запускает серверный сокет и при подключении клиента создаёт объект `ClientConnection`. В `ClientConnection::onReadyRead()` данные накапливаются в буфере, извлекаются по разделителю `\n`, парсится JSON и вызывается соответствующий метод `DBWorker`. Результат отправляется обратно в виде JSON. Сервер способен параллельно обрабатывать несколько клиентов.

1.4.4. Модуль логирования (Logs)

Класс `Logs` записывает сообщения с уровнями `INF`, `WARN`, `ERR` в файл и дублирует их в консоль. При недоступности основного файла использует резервный путь в системной папке приложения. Формат строки: `yyyy-MM-dd hh:mm:ss [УРОВЕНЬ] сообщение`.

1.4.5. Модуль настроек (Settings)

Класс `Settings` предоставляет `Q_PROPERTY` для доступа из QML. Параметры (хост, порт БД, сервер, режим, пути к логам и отчётам) сохраняются через `QSettings` в системном реестре/↔ конфигурационном файле. Методы `load()` и `save()` вызываются при запуске и при закрытии диалога настроек.

1.4.6. Пользовательский интерфейс (QML)

- **LoginPage.qml** – экран входа. Поля «Логин», «Пароль», кнопки «Войти» и «Настройки». При успехе испускает сигнал `loginSuccess`.
- **Main.qml** – главное окно. Панель инструментов с кнопками меню (☰) и синхронизации (↔), выдвигающее меню (`Drawer`), пункты которого зависят от роли пользователя.
- **OrderPage.qml** – страница заказа. Левая колонка – меню с фильтрами, правая – корзина. Кнопки «Добавить», «Удалить», «Оформить заказ».
- **StockPage.qml** – страница склада. Список продуктов, поле ввода количества, кнопки «Добавить», «Убавить», «Установить».
- **ReportsPage.qml** – страница отчётов. Выбор типа отчёта, фильтры (даты, блюдо, столик, официант), кнопка «Сформировать», вывод результатов и «Сохранить отчёт» (CSV).

1.5. Интеграция программных модулей

1.5.1. Подключаемые внешние модули

- Qt 6 (Core, Quick, Sql, Network)
- PostgreSQL client library (`libpq`) – для драйвера QPSQL.

1.5.2. Интеграция C++ и QML

Объекты регистрируются в контексте через `setContextProperty()`:

```
engine.rootContext()->setContextProperty("dbWorker", worker);
```

В QML методы вызываются напрямую (`dbWorker.checkUser(...)`). Сигналы (например, `errorOccurred`) перехватываются через `Connections`.

1.5.3. Клиент-серверное взаимодействие

Используется TCP. Каждое сообщение – JSON-строка, заканчивающаяся `\n`. Клиент нумерует запросы (`requestId`) и ожидает соответствующий ответ. Таймаут – 60 секунд. Сервер обрабатывает каждого клиента в отдельном объекте, что позволяет параллельно обслуживать множество подключений.

1.6. Руководство пользователя

1.6.1. Начальный запуск

1. Установите приложение на планшет (APK для Android или соберите под Windows).
2. При первом запуске откроется окно авторизации.
3. Если приложение работает в **клиентском режиме**, убедитесь, что **сервер** (другое устройство с программой в режиме сервера) активен и доступен по сети.
4. При необходимости настройте подключение через кнопку «Настройки».

1.6.2. Окно авторизации

- Поле «**Логин**» – введите имя пользователя.
- Поле «**Пароль**» – введите пароль.
- Кнопка «**Войти**» – проверка данных. При успехе открывается главное окно с соответствующими правами.
- Кнопка «**Настройки**» – открывает диалог конфигурации.

1.6.3. Диалог настроек

Группа «База данных» (видна только в серверном режиме): хост, порт, имя БД, пользователь, пароль; кнопка «Проверить подключение».

Группа «Сеть»: IP сервера, порт; флажок «Режим сервера»; кнопки «Проверить сервер», «Показать IP устройства».

Группа «Прочее»: путь к логам, папка для отчётов.

Сохранение – кнопки «ОК» (сохранить и закрыть) и «Отмена».

1.6.4. Главное окно и выдвижное меню

- **Верхняя панель**: кнопка ☰ (открыть меню), название «Ресторан», кнопка ☰ (синхронизация данных).
- **Выдвижное меню**:
 - Отображается ФИО пользователя.
 - Пункты зависят от роли: официант – «Оформление заказа»; кладовщик – «Склад»; администратор – «Оформление заказа», «Склад», «Отчёты».
 - Кнопки: «Настройки», «Выйти», «Закрыть приложение».
- Меню закрывается касанием вне его или повторным нажатием кнопки ☰.

1.6.5. Оформление заказа (страница для официанта)

Левая колонка – меню:

- Фильтр по категории (Супы, Горячее, Салаты, Напитки, Все)
- Флажок «Только доступные»
- Список блюд: название, цена, категория, описание, состав. Недоступные блюда помечены красным.

Правая колонка – заказ:

- «Текущий заказ» – корзина с добавленными блюдами.
- Поля: «Столик №» (1–50), «Кол-во» (1–10), «Комментарий».
- Кнопка «Добавить» – проверяет наличие ингредиентов. Если после списания остатки падают ниже порога, появляется предупреждение (добавление разрешается). При нехватке продуктов – ошибка.
- Кнопка «Удалить» – удаляет позицию из корзины и возвращает ингредиенты.
- Кнопка «Оформить заказ» – фиксирует заказ в БД, очищает корзину и обновляет меню.

1.6.6. Управление складом (страница для кладовщика)

- Список продуктов: название, текущее количество (кг, л, шт.). Продукты ниже минимального порога подсвечены розовым.
- Выбор продукта – касание строки.
- Поле «Количество» (целое число 0–10000).
- Кнопки: «Добавить» (увеличить остаток), «Убавить» (уменьшить, если результат ≥ 0), «Установить» (задать точное значение). Изменения немедленно синхронизируются с сервером.

1.6.7. Отчёты (страница для администратора)

Типы отчётов (выпадающий список):

1. **Выручка** – фильтры: дата «с», «по», блюдо, столик, официант. Выводится общая выручка и разбивка по дням.
2. **Популярность блюд** – фильтры: дата «с», «по». Результат – список блюд с количеством проданных порций, отсортированный по убыванию.
3. **Малые остатки** – без фильтров. Отображаются продукты, у которых остаток ниже порога.

Сохранение отчёта: после формирования нажмите «Сохранить отчёт», выберите имя файла. Файл будет создан в папке, указанной в настройках (по умолчанию reports). Формат – CSV (разделитель запятая, кодировка UTF-8).

1.7. Настройки и системные файлы

- **Файл настроек** – хранится через QSettings (реестр Windows или конфигурационный файл). Вручную не редактировать.
- **Лог-файл** – по умолчанию logs.txt в рабочей папке. При ошибке записи используется системная папка приложения.
- **Папка отчётов** – задаётся в диалоге настроек, используется для сохранения CSV-файлов.

1.8. Сборка проекта (для разработчиков)

Требования:

- Qt 6.8 или новее (Core, Quick, Sql, Network)
- CMake ≥ 3.16
- PostgreSQL (серверный режим) либо клиентская часть без БД.

Шаги сборки:

```
git clone <url>
cd RestN
mkdir build && cd build
cmake ..
cmake --build . --config Release
```

Запуск на Android: выберите в Qt Creator комплект Android, соберите APK, установите на планшет. На серверном устройстве должно быть установлено приложение (режим сервера) или отдельный PostgreSQL.

1.9. Контакты

Автор: Самсыка Родион Михайлович

Группа: И-42

Специальность: 09.02.07 Информационные системы и программирование

Научный руководитель: Фомин Андрей Томасович

Год: 2026

Глава 2

Hierarchical Index

2.1. Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

std::exception	
Exceptions	21
DatabaseEx	14
FileEx	22
NetworkEx	23
ValidateEx	34
QObject	
ClientConnection	13
DBWorker	14
Logs	22
Settings	24
TcpClient	25
TcpServer	33

Глава 3

Class Index

3.1. Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ClientConnection	Обработчик одного клиентского TCP-соединения	13
DatabaseEx	Исключение, связанное с ошибками базы данных	14
DBWorker	Класс для взаимодействия с базой данных PostgreSQL ресторана	14
Exceptions	Базовый класс исключений, принимающий QString	21
FileEx	Исключение, связанное с файловыми операциями	22
Logs	Сервис логирования сообщений в файл	22
NetworkEx	Исключение, связанное с сетевыми ошибками	23
Settings	Класс для хранения и управления настройками приложения	24
TcpClient	Клиент для удалённого взаимодействия с сервером приложения	25
TcpServer	TCP-сервер, принимающий подключения от клиентов	33
ValidateEx	Исключение валидации (некорректные данные, нехватка продуктов и т.п.) . . .	34

Глава 4

File Index

4.1. File List

Here is a list of all documented files with brief descriptions:

C:/Users/rodio/Desktop/W/Diplom/app/include/ dbworker.h	37
C:/Users/rodio/Desktop/W/Diplom/app/include/ exceptions.h	38
C:/Users/rodio/Desktop/W/Diplom/app/include/ logs.h	38
C:/Users/rodio/Desktop/W/Diplom/app/include/ settings.h	39
C:/Users/rodio/Desktop/W/Diplom/app/include/ tcpclient.h	40
C:/Users/rodio/Desktop/W/Diplom/app/include/ tcpserver.h	41

Глава 5

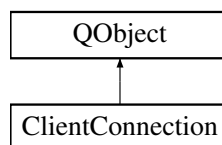
Class Documentation

5.1. ClientConnection Class Reference

Обработчик одного клиентского TCP-соединения.

```
#include <tcpserver.h>
```

Inheritance diagram for ClientConnection:



Public Member Functions

- [ClientConnection](#) ([Logs](#) *logger, qintptr socketDescriptor, [DBWorker](#) *dbWorker, QObject *parent=nullptr)

Конструктор.

5.1.1. Detailed Description

Обработчик одного клиентского TCP-соединения.

Принимает JSON-команды, вызывает соответствующие методы [DBWorker](#) и отправляет обратно ответ.

5.1.2. Constructor & Destructor Documentation

5.1.2.1. ClientConnection()

```
ClientConnection::ClientConnection (  
    Logs * logger,  
    qintptr socketDescriptor,  
    DBWorker * dbWorker,  
    QObject * parent = nullptr)
```

Конструктор.

Parameters

<i>logger</i>	Логгер.
<i>socketDescriptor</i>	Дескриптор сокета.
<i>dbWorker</i>	Указатель на объект работы с БД.

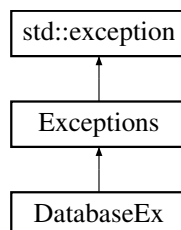
<i>parent</i>	Родительский объект.
---------------	----------------------

5.2. DatabaseEx Class Reference

Исключение, связанное с ошибками базы данных.

```
#include <exceptions.h>
```

Inheritance diagram for DatabaseEx:



Public Member Functions

- **DatabaseEx** (const QString &message)

Public Member Functions inherited from [Exceptions](#)

- [Exceptions](#) (const QString &message)
Конструктор.
- virtual const char * **what** () const noexcept override

5.2.1. Detailed Description

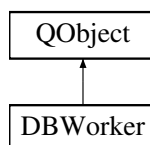
Исключение, связанное с ошибками базы данных.

5.3. DBWorker Class Reference

Класс для взаимодействия с базой данных PostgreSQL ресторана.

```
#include <dbworker.h>
```

Inheritance diagram for DBWorker:



Signals

- void **errorOccurred** (const QString &message)
Сигнал, передающий текст ошибки для отображения в интерфейсе.
- void **foodAvailabilityChanged** ()
Сообщает об изменении доступности блюд (например, после обновления остатков).

Public Member Functions

- [DBWorker](#) (Logs *logger, QObject *parent=nullptr)
Конструктор.
- Q_INVOKABLE bool [checkUser](#) (const QString &login, const QString &password)

- Проверяет логин и пароль пользователя.*
- Q_INVOKABLE QString **getUserName** () const
Возвращает имя текущего пользователя.
- Q_INVOKABLE QString **getUserRole** () const
Возвращает роль текущего пользователя.
- Q_INVOKABLE int **getUserId** () const
Возвращает идентификатор текущего пользователя.
- Q_INVOKABLE bool **checkAndConsume** (int foodId, int quantity)
Проверяет наличие ингредиентов и списывает их со склада.
- Q_INVOKABLE bool **returnIngredients** (int foodId, int quantity)
Возвращает ингредиенты на склад при отмене позиции заказа.
- Q_INVOKABLE bool **refreshAllFoodsAvailability** ()
Пересчитывает доступность всех блюд на основе остатков продуктов.
- Q_INVOKABLE bool **reconnect** (const QString &host, int port, const QString &dbName, const QString &user, const QString &password)
Переподключает базу данных с новыми параметрами.
- Q_INVOKABLE QStringList **getLocalIps** ()
Возвращает список локальных IPv4-адресов.
- Q_INVOKABLE bool **pingServer** (const QString &host, int port)
Проверяет доступность удалённого сервера (заглушка, всегда true).
- Q_INVOKABLE QVariantList **getFoods** ()
Загружает меню с составом и доступностью.
- Q_INVOKABLE QVariantList **getProducts** ()
Загружает складские остатки продуктов.
- Q_INVOKABLE QVariantList **getDishesList** ()
Возвращает краткий список блюд (ID и название).
- Q_INVOKABLE QVariantList **getWaitersList** ()
Возвращает список официантов.
- Q_INVOKABLE bool **updateProductQuantity** (int productId, int newQuantity)
Устанавливает новое абсолютное количество продукта.
- Q_INVOKABLE bool **addProductQuantity** (int productId, int delta)
Изменяет количество продукта на заданную величину.
- Q_INVOKABLE int **createOrder** (int tableNumber, int userId, const QString &specialRequests, double totalCost)
Создаёт новый заказ.
- Q_INVOKABLE bool **addOrderItem** (int orderId, int foodId, int quantity, double price)
Добавляет позицию в заказ.
- Q_INVOKABLE QVariantMap **getReportRevenue** (const QString &from, const QString &to, int dishId, int tableNumber, int waiterId)
Возвращает общую выручку за период с применёнными фильтрами.
- Q_INVOKABLE QVariantList **getRevenueByDay** (const QString &from, const QString &to, int dishId, int tableNumber, int waiterId)
Возвращает выручку, сгруппированную по дням.
- Q_INVOKABLE QVariantList **getFoodPopularity** (const QString &from, const QString &to)
Получает популярность блюд за период.
- Q_INVOKABLE QVariantList **getReportLowStock** ()
Возвращает продукты, количество которых ниже минимального порога.
- Q_INVOKABLE QVariantList **getLowStockWarning** (int foodId, int quantity)
Предупреждает о продуктах, остаток которых станет ниже порога после приготовления блюда.
- Q_INVOKABLE bool **saveReportToCSV** (const QString &reportType, const QString &from, const QString &to, int dishId, int tableNumber, int waiterId, const QString &fileName)
Сохраняет отчёт в CSV-файл.

5.3.1. Detailed Description

Класс для взаимодействия с базой данных PostgreSQL ресторана. Предоставляет методы аутентификации, работы с меню, складом, заказами и отчётами. Хранит информацию о текущем пользователе.

5.3.2. Constructor & Destructor Documentation

5.3.2.1. DBWorker()

```
DBWorker::DBWorker (  
    Logs * logger,  
    QObject * parent = nullptr)
```

Конструктор.

Parameters

<i>logger</i>	Указатель на объект логирования.
<i>parent</i>	Родительский объект (по умолчанию nullptr).

5.3.3. Member Function Documentation

5.3.3.1. addOrderItem()

```
bool DBWorker::addOrderItem (  
    int orderId,  
    int foodId,  
    int quantity,  
    double price)
```

Добавляет позицию в заказ.

Parameters

<i>orderId</i>	ID заказа.
<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество.
<i>price</i>	Цена за единицу.

Returns

true при успешной вставке.

5.3.3.2. addProductQuantity()

```
bool DBWorker::addProductQuantity (  
    int productId,  
    int delta)
```

Изменяет количество продукта на заданную величину.

Parameters

<i>productId</i>	ID продукта.
<i>delta</i>	Изменение (может быть отрицательным).

Returns

true при успехе.

5.3.3.3. checkAndConsume()

```
bool DBWorker::checkAndConsume (  
    int foodId,  
    int quantity)
```

Проверяет наличие ингредиентов и списывает их со склада.

Parameters

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Returns

true, если списание прошло успешно.

5.3.3.4. checkUser()

```
bool DBWorker::checkUser (  
    const QString & login,  
    const QString & password)
```

Проверяет логин и пароль пользователя.

Parameters

<i>login</i>	Логин.
<i>password</i>	Пароль.

Returns

true при успешной аутентификации.

5.3.3.5. createOrder()

```
int DBWorker::createOrder (  
    int tableNumber,  
    int userId,  
    const QString & specialRequests,  
    double totalCost)
```

Создаёт новый заказ.

Parameters

<i>tableNumber</i>	Номер столика.
<i>userId</i>	ID официанта.
<i>specialRequests</i>	Особые пожелания.
<i>totalCost</i>	Итоговая стоимость.

Returns

ID созданного заказа или -1 при ошибке.

5.3.3.6. `getFoodPopularity()`

```
QVariantList DBWorker::getFoodPopularity (  
    const QString & from,  
    const QString & to)
```

Получает популярность блюд за период.

Parameters

<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.

Returns

Список словарей с ключами "foodName" и "count".

5.3.3.7. `getLowStockWarning()`

```
QVariantList DBWorker::getLowStockWarning (  
    int foodId,  
    int quantity)
```

Предупреждает о продуктах, остаток которых станет ниже порога после приготовления блюда.

Parameters

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Returns

Список предупреждений с полями `productName`, `currentQuantity`, `newQuantity`, `minThreshold`.

5.3.3.8. `getReportRevenue()`

```
QVariantMap DBWorker::getReportRevenue (  
    const QString & from,  
    const QString & to,  
    int dishId,  
    int tableNumber,  
    int waiterId)
```

Возвращает общую выручку за период с применёнными фильтрами.

Parameters

<i>from</i>	Начальная дата (ГГГГ-ММ-ДД).
<i>to</i>	Конечная дата.
<i>dishId</i>	ID блюда (0 – без фильтра).
<i>tableNumber</i>	Номер столика (0 – без фильтра).

<i>waiterId</i>	ID официанта (0 – без фильтра).
-----------------	---------------------------------

Returns

Словарь с ключом "totalRevenue" или "error".

5.3.3.9. getRevenueByDay()

```
QVariantList DBWorker::getRevenueByDay (  
    const QString & from,  
    const QString & to,  
    int dishId,  
    int tableNumber,  
    int waiterId)
```

Возвращает выручку, сгруппированную по дням.

Parameters

<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.
<i>dishId</i>	Фильтр по блюду (0 – без фильтра).
<i>tableNumber</i>	Фильтр по столику (0 – без фильтра).
<i>waiterId</i>	Фильтр по официанту (0 – без фильтра).

Returns

Список словарей с полями "date" и "total".

5.3.3.10. pingServer()

```
bool DBWorker::pingServer (  
    const QString & host,  
    int port)
```

Проверяет доступность удалённого сервера (заглушка, всегда true).

Parameters

<i>host</i>	Адрес.
<i>port</i>	Порт.

Returns

true.

5.3.3.11. reconnect()

```
bool DBWorker::reconnect (  
    const QString & host,  
    int port,  
    const QString & dbName,
```

```
const QString & user,  
const QString & password)
```

Переподключает базу данных с новыми параметрами.

Parameters

<i>host</i>	Хост.
<i>port</i>	Порт.
<i>dbName</i>	Имя базы данных.
<i>user</i>	Пользователь.
<i>password</i>	Пароль.

Returns

true при успешном подключении.

5.3.3.12. returnIngredients()

```
bool DBWorker::returnIngredients (  
    int foodId,  
    int quantity)
```

Возвращает ингредиенты на склад при отмене позиции заказа.

Parameters

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Returns

true в случае успеха.

5.3.3.13. saveReportToCSV()

```
bool DBWorker::saveReportToCSV (  
    const QString & reportType,  
    const QString & from,  
    const QString & to,  
    int dishId,  
    int tableNumber,  
    int waiterId,  
    const QString & fileName)
```

Сохраняет отчёт в CSV-файл.

Parameters

<i>reportType</i>	Тип отчёта ("revenue", "popularity", "lowstock").
<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.
<i>dishId</i>	Фильтр по блюду.
<i>tableNumber</i>	Фильтр по столику.

<i>waiterId</i>	Фильтр по официанту.
<i>fileName</i>	Полный путь к выходному файлу.

Returns

true при успешном сохранении.

5.3.3.14. updateProductQuantity()

```
bool DBWorker::updateProductQuantity (
    int productId,
    int newQuantity)
```

Устанавливает новое абсолютное количество продукта.

Parameters

<i>productId</i>	ID продукта.
<i>newQuantity</i>	Новое количество.

Returns

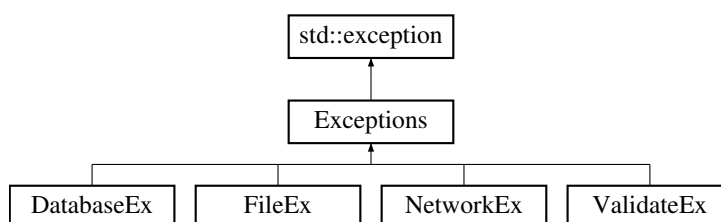
true при успешном обновлении.

5.4. Exceptions Class Reference

Базовый класс исключений, принимающий QString.

```
#include <exceptions.h>
```

Inheritance diagram for Exceptions:



Public Member Functions

- [Exceptions](#) (const QString &message)
Конструктор.
- virtual const char * **what** () const noexcept override

5.4.1. Detailed Description

Базовый класс исключений, принимающий QString.

Хранит сообщение в std::string для совместимости с std::exception. Все специализированные исключения наследуют этот класс.

5.4.2. Constructor & Destructor Documentation

5.4.2.1. Exceptions()

Exceptions::Exceptions (
 const QString & message) [explicit]
 Конструктор.

Parameters

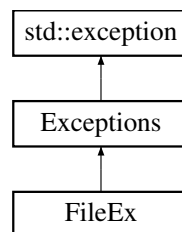
<i>message</i>	Описание ошибки.
----------------	------------------

5.5. FileEx Class Reference

Исключение, связанное с файловыми операциями.

```
#include <exceptions.h>
```

Inheritance diagram for FileEx:



Public Member Functions

- **FileEx** (const QString &message)

Public Member Functions inherited from [Exceptions](#)

- [Exceptions](#) (const QString &message)
 Конструктор.
- virtual const char * **what** () const noexcept override

5.5.1. Detailed Description

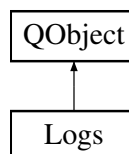
Исключение, связанное с файловыми операциями.

5.6. Logs Class Reference

Сервис логирования сообщений в файл.

```
#include <logs.h>
```

Inheritance diagram for Logs:



Public Member Functions

- **Logs** (QObject *parent=nullptr)

- Q_INVOKABLE void **info** (const QString &msg)
Записывает информационное сообщение.
- Q_INVOKABLE void **warning** (const QString &msg)
Записывает предупреждение.
- Q_INVOKABLE void **error** (const QString &msg)
Записывает сообщение об ошибке.
- Q_INVOKABLE void [setLogPath](#) (const QString &path)
Устанавливает путь к лог-файлу и переперезывает его.

5.6.1. Detailed Description

Сервис логирования сообщений в файл.

Записывает сообщения с меткой времени и уровнем (INF, WARN, ERR). Если основной файл не может быть открыт, использует резервный путь в директории AppLocalDataLocation.

5.6.2. Member Function Documentation

5.6.2.1. setLogPath()

```
void Logs::setLogPath (
    const QString & path)
```

Устанавливает путь к лог-файлу и переперезывает его.

Parameters

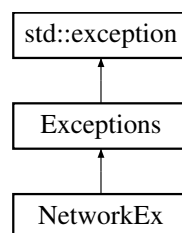
<i>path</i>	Новый путь.
-------------	-------------

5.7. NetworkEx Class Reference

Исключение, связанное с сетевыми ошибками.

```
#include <exceptions.h>
```

Inheritance diagram for NetworkEx:



Public Member Functions

- **NetworkEx** (const QString &message)

Public Member Functions inherited from [Exceptions](#)

- [Exceptions](#) (const QString &message)
Конструктор.
- virtual const char * **what** () const noexcept override

5.7.1. Detailed Description

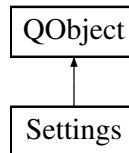
Исключение, связанное с сетевыми ошибками.

5.8. Settings Class Reference

Класс для хранения и управления настройками приложения.

```
#include <settings.h>
```

Inheritance diagram for Settings:



Signals

- void **dbHostChanged** ()
- void **dbPortChanged** ()
- void **dbNameChanged** ()
- void **dbUserChanged** ()
- void **dbPasswordChanged** ()
- void **isServerModeChanged** ()
- void **logPathChanged** ()
- void **serverHostChanged** ()
- void **serverPortChanged** ()
- void **reportsPathChanged** ()

Public Member Functions

- [Settings](#) ([Logs](#) *logger, QObject *parent=nullptr)
Конструктор. Автоматически вызывает [load\(\)](#).
- QString **dbHost** () const
- void **setDbHost** (const QString &host)
Устанавливает хост БД.
- int **dbPort** () const
- void **setDbPort** (int port)
Устанавливает порт БД.
- QString **dbName** () const
- void **setDbName** (const QString &name)
Устанавливает имя БД.
- QString **dbUser** () const
- void **setDbUser** (const QString &user)
Устанавливает пользователя БД.
- QString **dbPassword** () const
- void **setDbPassword** (const QString &password)
Устанавливает пароль БД.
- bool **isServerMode** () const
- void **setIsServerMode** (bool mode)
Включает/выключает серверный режим.
- QString **logPath** () const
- void **setLogPath** (const QString &path)
Задаёт путь к лог-файлу.
- QString **serverHost** () const
- void **setServerHost** (const QString &host)
Устанавливает IP-адрес сервера.
- int **serverPort** () const

- void **setServerPort** (int port)
Устанавливает порт сервера.
- QString **reportsPath** () const
- void **setReportsPath** (const QString &path)
Задаёт каталог для сохранения отчётов.
- Q_INVOKABLE void **load** ()
Загружает все настройки из QSettings.
- Q_INVOKABLE void **save** ()
Сохраняет текущие настройки в QSettings.

Properties

- QString **dbHost**
- int **dbPort**
- QString **dbName**
- QString **dbUser**
- QString **dbPassword**
- bool **isServerMode**
- QString **logPath**
- QString **serverHost**
- int **serverPort**
- QString **reportsPath**

5.8.1. Detailed Description

Класс для хранения и управления настройками приложения.
Обёртка над QSettings, предоставляющая свойства QML для доступа к параметрам базы данных, сети и путям.

5.8.2. Constructor & Destructor Documentation

5.8.2.1. Settings()

```
Settings::Settings (
    Logs * logger,
    QObject * parent = nullptr) [explicit]
```

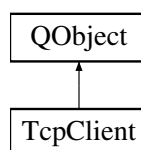
Конструктор. Автоматически вызывает [load\(\)](#).

Parameters

<i>logger</i>	Указатель на логгер.
<i>parent</i>	Родительский объект.

5.9. TcpClient Class Reference

Клиент для удалённого взаимодействия с сервером приложения.
#include <tcpclient.h>
Inheritance diagram for TcpClient:



Signals

- void **errorOccurred** (const QString &message)
Сообщение об ошибке для UI.
- void **dataChanged** ()
Уведомление от сервера об изменении данных (data_changed).
- void **responseReceived** (int requestId, bool success, const QVariantMap &data)
Пришёл ответ на ранее отправленный запрос.

Public Member Functions

- [TcpClient](#) ([Logs](#) *logger, QObject *parent=nullptr)
Конструктор.
- bool [connectToServer](#) (const QString &host, quint16 port)
Подключается к серверу.
- void **disconnectFromServer** ()
Закрывает соединение и очищает буфер.
- bool [isConnected](#) () const
- Q_INVOKABLE bool [checkUser](#) (const QString &login, const QString &password)
Отправляет на сервер запрос аутентификации.
- Q_INVOKABLE QString [getUserName](#) () const
- Q_INVOKABLE QString [getUserRole](#) () const
- Q_INVOKABLE int [getUserId](#) () const
- Q_INVOKABLE QVariantList [getFoods](#) ()
Запрашивает меню с сервера.
- Q_INVOKABLE QVariantList [getProducts](#) ()
Запрашивает список продуктов.
- Q_INVOKABLE bool [updateProductQuantity](#) (int productId, int newQuantity)
Отправляет команду установки абсолютного количества продукта.
- Q_INVOKABLE bool [addProductQuantity](#) (int productId, int delta)
Отправляет команду изменения количества продукта на дельту.
- Q_INVOKABLE int [createOrder](#) (int tableNumber, int userId, const QString &specialRequests, double totalCost)
Создаёт заказ на сервере.
- Q_INVOKABLE bool [addOrderItem](#) (int orderId, int foodId, int quantity, double price)
Добавляет позицию заказа.
- Q_INVOKABLE bool [checkAndConsume](#) (int foodId, int quantity)
Запрашивает проверку и списание ингредиентов.
- Q_INVOKABLE bool [returnIngredients](#) (int foodId, int quantity)
Запрашивает возврат ингредиентов на склад.
- Q_INVOKABLE QVariantList [getLowStockWarning](#) (int foodId, int quantity)
Получает предупреждения о снижении остатков ниже порога.
- Q_INVOKABLE QVariantMap [getReportRevenue](#) (const QString &from, const QString &to, int dishId, int tableNumber, int waiterId)
Запрашивает отчёт по общей выручке.
- Q_INVOKABLE QVariantList [getRevenueByDay](#) (const QString &from, const QString &to, int dishId, int tableNumber, int waiterId)
Запрашивает выручку по дням.
- Q_INVOKABLE QVariantList [getFoodPopularity](#) (const QString &from, const QString &to)
Запрашивает популярность блюд.
- Q_INVOKABLE QVariantList [getReportLowStock](#) ()
Запрашивает отчёт по продуктам с низким остатком.

- Q_INVOKABLE QVariantList **getDishesList** ()
Запрашивает краткий список блюд (ID и название).
- Q_INVOKABLE QVariantList **getWaitersList** ()
Запрашивает список официантов.
- Q_INVOKABLE QStringList **getLocalIps** ()
Возвращает IP-адреса локального устройства.
- Q_INVOKABLE bool **pingServer** (const QString &host, int port)
Проверяет доступность удалённого хоста через пробное TCP-подключение.
- Q_INVOKABLE bool **reconnect** (const QString &host, quint16 port)
Переподключается к серверу (сначала отключается, затем подключается заново).
- Q_INVOKABLE bool **saveReportToCSV** (const QString &reportType, const QString &from, const QString &to, int dishId, int tableNumber, int waiterId, const QString &fileName)
Отправляет запрос на сохранение отчёта в CSV на сервер.

5.9.1. Detailed Description

Клиент для удалённого взаимодействия с сервером приложения. Устанавливает TCP-соединение, обменивается JSON-сообщениями. Дублирует интерфейс [DBWorker](#) для прозрачной работы из QML независимо от режима (локальный или клиент-сервер). При потере связи генерирует сигнал `errorOccurred`.

5.9.2. Constructor & Destructor Documentation

5.9.2.1. TcpClient()

```
TcpClient::TcpClient (
    Logs * logger,
    QObject * parent = nullptr) [explicit]
```

Конструктор.

Parameters

<i>logger</i>	Указатель на логгер.
<i>parent</i>	Родительский объект.

5.9.3. Member Function Documentation

5.9.3.1. addOrderItem()

```
bool TcpClient::addOrderItem (
    int orderId,
    int foodId,
    int quantity,
    double price)
```

Добавляет позицию заказа.

Parameters

<i>orderId</i>	ID заказа.
<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество.
<i>price</i>	Цена.

Returns

true при успехе.

5.9.3.2. addProductQuantity()

```
bool TcpClient::addProductQuantity (  
    int productId,  
    int delta)
```

Отправляет команду изменения количества продукта на дельту.

Parameters

<i>productId</i>	ID продукта.
<i>delta</i>	Изменение (может быть отрицательным).

Returns

true при успехе.

5.9.3.3. checkAndConsume()

```
bool TcpClient::checkAndConsume (  
    int foodId,  
    int quantity)
```

Запрашивает проверку и списание ингредиентов.

Parameters

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Returns

true, если списание разрешено и выполнено.

5.9.3.4. checkUser()

```
bool TcpClient::checkUser (  
    const QString & login,  
    const QString & password)
```

Отправляет на сервер запрос аутентификации.

Parameters

<i>login</i>	Логин.
<i>password</i>	Пароль.

Returns

true при успешной проверке.

5.9.3.5. connectToServer()

```
bool TcpClient::connectToServer (  
    const QString & host,  
    quint16 port)
```

Подключается к серверу.

Parameters

<i>host</i>	Адрес сервера.
<i>port</i>	Порт.

Returns

true, если соединение установлено.

5.9.3.6. createOrder()

```
int TcpClient::createOrder (  
    int tableNumber,  
    int userId,  
    const QString & specialRequests,  
    double totalCost)
```

Создаёт заказ на сервере.

Parameters

<i>tableNumber</i>	Номер столика.
<i>userId</i>	ID пользователя.
<i>specialRequests</i>	Комментарий.
<i>totalCost</i>	Общая стоимость.

Returns

ID созданного заказа или -1.

5.9.3.7. getFoodPopularity()

```
QVariantList TcpClient::getFoodPopularity (  
    const QString & from,  
    const QString & to)
```

Запрашивает популярность блюд.

Parameters

<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.

Returns

Список словарей с ключами "foodName" и "count".

5.9.3.8. `getFoods()`

`QVariantList TcpClient::getFoods ()`

Запрашивает меню с сервера.

Returns

Список блюд (`QVariantList`).

5.9.3.9. `getLowStockWarning()`

`QVariantList TcpClient::getLowStockWarning (`

`int foodId,`

`int quantity)`

Получает предупреждения о снижении остатков ниже порога.

Parameters

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Returns

Список предупреждений.

5.9.3.10. `getProducts()`

`QVariantList TcpClient::getProducts ()`

Запрашивает список продуктов.

Returns

Список продуктов.

5.9.3.11. `getReportRevenue()`

`QVariantMap TcpClient::getReportRevenue (`

`const QString & from,`

`const QString & to,`

`int dishId,`

`int tableNumber,`

`int waiterId)`

Запрашивает отчёт по общей выручке.

Parameters

<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.
<i>dishId</i>	ID блюда (0 – все).
<i>tableNumber</i>	№ столика (0 – все).
<i>waiterId</i>	ID официанта (0 – все).

Returns

Словарь с ключом "totalRevenue" или "error".

5.9.3.12. getRevenueByDay()

```
QVariantList TcpClient::getRevenueByDay (  
    const QString & from,  
    const QString & to,  
    int dishId,  
    int tableNumber,  
    int waiterId)
```

Запрашивает выручку по дням.

Parameters

<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.
<i>dishId</i>	ID блюда.
<i>tableNumber</i>	№ столика.
<i>waiterId</i>	ID официанта.

Returns

Список словарей с ключами "date" и "total".

5.9.3.13. getUserId()

```
int TcpClient::getUserId () const
```

Returns

ID текущего пользователя.

5.9.3.14. getUsername()

```
QString TcpClient::getUserName () const
```

Returns

Имя текущего пользователя.

5.9.3.15. getUserRole()

```
QString TcpClient::getUserRole () const
```

Returns

Роль текущего пользователя.

5.9.3.16. isConnected()

```
bool TcpClient::isConnected () const
```

Returns

true, если сокет находится в состоянии ConnectedState.

5.9.3.17. pingServer()

```
bool TcpClient::pingServer (  
    const QString & host,  
    int port)
```

Проверяет доступность удалённого хоста через пробное TCP-подключение.

Parameters

<i>host</i>	Адрес.
<i>port</i>	Порт.

Returns

true, если соединение удалось установить за 2 с.

5.9.3.18. reconnect()

```
bool TcpClient::reconnect (  
    const QString & host,  
    quint16 port)
```

Переподключается к серверу (сначала отключается, затем подключается заново).

Parameters

<i>host</i>	Новый адрес.
<i>port</i>	Новый порт.

Returns

true при успешном подключении.

5.9.3.19. returnIngredients()

```
bool TcpClient::returnIngredients (  
    int foodId,  
    int quantity)
```

Запрашивает возврат ингредиентов на склад.

Parameters

<i>foodId</i>	ID блюда.
<i>quantity</i>	Количество порций.

Returns

true при успехе.

5.9.3.20. saveReportToCSV()

```
bool TcpClient::saveReportToCSV (  
    const QString & reportType,  
    const QString & from,  
    const QString & to,
```

```
int dishId,
int tableNumber,
int waiterId,
const QString & fileName)
```

Отправляет запрос на сохранение отчёта в CSV на сервер.

Parameters

<i>reportType</i>	Тип отчёта.
<i>from</i>	Начальная дата.
<i>to</i>	Конечная дата.
<i>dishId</i>	ID блюда (0 – все).
<i>tableNumber</i>	№ столика (0 – все).
<i>waiterId</i>	ID официанта (0 – все).
<i>fileName</i>	Полный путь к сохраняемому файлу.

Returns

true, если сервер успешно создал файл.

5.9.3.21. updateProductQuantity()

```
bool TcpClient::updateProductQuantity (
    int productId,
    int newQuantity)
```

Отправляет команду установки абсолютного количества продукта.

Parameters

<i>productId</i>	ID продукта.
<i>newQuantity</i>	Новое количество.

Returns

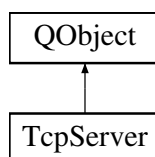
true при успехе.

5.10. TcpServer Class Reference

TCP-сервер, принимающий подключения от клиентов.

```
#include <tcpserver.h>
```

Inheritance diagram for TcpServer:



Public Member Functions

- [TcpServer](#) ([Logs](#) *logger, [DBWorker](#) *dbWorker, QObject *parent=nullptr)

Конструктор.

- bool **start** (quint16 port)

Запускает прослушивание порта.

- void **stop** ()

Останавливает сервер и удаляет все соединения.

5.10.1. Detailed Description

TCP-сервер, принимающий подключения от клиентов.

При новом подключении создаёт [ClientConnection](#) и передаёт ему [DBWorker](#).

5.10.2. Constructor & Destructor Documentation

5.10.2.1. TcpServer()

```
TcpServer::TcpServer (
    Logs * logger,
    DBWorker * dbWorker,
    QObject * parent = nullptr) [explicit]
```

Конструктор.

Parameters

<i>logger</i>	Логгер.
<i>dbWorker</i>	Указатель на DBWorker .
<i>parent</i>	Родительский объект.

5.10.3. Member Function Documentation

5.10.3.1. start()

```
bool TcpServer::start (
    quint16 port)
```

Запускает прослушивание порта.

Parameters

<i>port</i>	Порт.
-------------	-------

Returns

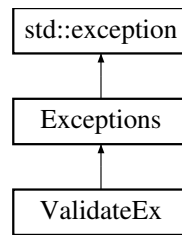
true при успехе.

5.11. ValidateEx Class Reference

Исключение валидации (некорректные данные, нехватка продуктов и т.п.).

```
#include <exceptions.h>
```

Inheritance diagram for ValidateEx:



Public Member Functions

- **ValidateEx** (const QString &message)

Public Member Functions inherited from [Exceptions](#)

- [Exceptions](#) (const QString &message)
Конструктор.
- virtual const char * **what** () const noexcept override

5.11.1. Detailed Description

Исключение валидации (некорректные данные, нехватка продуктов и т.п.).

Глава 6

File Documentation

6.1. dbworker.h

```
00001 #ifndef DBWORKER_H
00002 #define DBWORKER_H
00003
00004 #include <QObject>
00005 #include <QSqlDatabase>
00006 #include <QSqlQuery>
00007 #include <QSqlError>
00008 #include <QVariant>
00009 #include <QVariantList>
00010 #include <QVariantMap>
00011 #include <QStringConverter>
00012 #include "logs.h"
00013 #include "exceptions.h"
00014
00021 class DBWorker : public QObject
00022 {
00023     Q_OBJECT
00024 public:
00030     DBWorker(Logs *logger, QObject *parent = nullptr);
00031     ~DBWorker();
00032
00039     Q_INVOKABLE bool checkUser(const QString &login, const QString &password);
00040
00042     Q_INVOKABLE QString getUsername() const { return userName; }
00044     Q_INVOKABLE QString getUserRole() const { return userRole; }
00046     Q_INVOKABLE int getUserId() const { return userId; }
00047
00054     Q_INVOKABLE bool checkAndConsume(int foodId, int quantity);
00055
00062     Q_INVOKABLE bool returnIngredients(int foodId, int quantity);
00063
00065     Q_INVOKABLE bool refreshAllFoodsAvailability();
00066
00076     Q_INVOKABLE bool reconnect(const QString &host, int port, const QString &dbName, const QString
00077 &user, const QString &password);
00077
00079     Q_INVOKABLE QStringList getLocalIps();
00080
00087     Q_INVOKABLE bool pingServer(const QString &host, int port);
00088
00090     Q_INVOKABLE QVariantList getFoods();
00091
00093     Q_INVOKABLE QVariantList getProducts();
00094
00096     Q_INVOKABLE QVariantList getDishesList();
00097
00099     Q_INVOKABLE QVariantList getWaitersList();
00100
00107     Q_INVOKABLE bool updateProductQuantity(int productId, int newQuantity);
00108
00115     Q_INVOKABLE bool addProductQuantity(int productId, int delta);
00116
00125     Q_INVOKABLE int createOrder(int tableNumber, int userId, const QString &specialRequests, double
00126 totalCost);
00135     Q_INVOKABLE bool addOrderItem(int orderId, int foodId, int quantity, double price);
00136
00146     Q_INVOKABLE QVariantMap getReportRevenue(const QString &from, const QString &to,
00147 int dishId, int tableNumber, int waiterId);
00148
00158     Q_INVOKABLE QVariantList getRevenueByDay(const QString &from, const QString &to,
```

```

00159             int dishId, int tableNumber, int waiterId);
00160
00167     Q_INVOKABLE QVariantList getFoodPopularity(const QString &from, const QString &to);
00168
00170     Q_INVOKABLE QVariantList getReportLowStock();
00171
00178     Q_INVOKABLE QVariantList getLowStockWarning(int foodId, int quantity);
00179
00191     Q_INVOKABLE bool saveReportToCSV(const QString &reportType,
00192                                     const QString &from,
00193                                     const QString &to,
00194                                     int dishId,
00195                                     int tableNumber,
00196                                     int waiterId,
00197                                     const QString &fileName);
00198
00199 signals:
00201     void errorOccurred(const QString &message);
00203     void foodAvailabilityChanged();
00204
00205 private:
00206     QSqlDatabase db;
00207     QString userName;
00208     QString userRole;
00209     int userId;
00210     Logs *logger;
00211 };
00212
00213 #endif // DBWORKER_H

```

6.2. exceptions.h

```

00001 #ifndef EXCEPTIONS_H
00002 #define EXCEPTIONS_H
00003
00004 #include <QString>
00005 #include <stdexcept>
00006
00013 class Exceptions : public std::exception
00014 {
00015 public:
00020     explicit Exceptions(const QString &message);
00021     virtual const char* what() const noexcept override;
00022
00023 private:
00024     std::string msg;
00025 };
00026
00028 class ValidateEx : public Exceptions
00029 {
00030 public:
00031     explicit ValidateEx(const QString &message) : Exceptions(message) {}
00032 };
00033
00035 class DatabaseEx : public Exceptions
00036 {
00037 public:
00038     explicit DatabaseEx(const QString &message) : Exceptions(message) {}
00039 };
00040
00042 class NetworkEx : public Exceptions
00043 {
00044 public:
00045     explicit NetworkEx(const QString &message) : Exceptions(message) {}
00046 };
00047
00049 class FileEx : public Exceptions
00050 {
00051 public:
00052     explicit FileEx(const QString &message) : Exceptions(message) {}
00053 };
00054
00055 #endif // EXCEPTIONS_H

```

6.3. logs.h

```

00001 #ifndef LOGS_H
00002 #define LOGS_H
00003
00004 #include <QObject>
00005 #include <QFile>

```

```

00006 #include <QTextStream>
00007 #include "exceptions.h"
00008
00016 class Logs : public QObject
00017 {
00018     Q_OBJECT
00019 public:
00020     explicit Logs(QObject *parent = nullptr);
00021     ~Logs();
00022
00024     Q_INVOKABLE void info(const QString &msg);
00026     Q_INVOKABLE void warning(const QString &msg);
00028     Q_INVOKABLE void error(const QString &msg);
00033     Q_INVOKABLE void setLogPath(const QString &path);
00034
00035 private:
00036     void writeLog(const QString &level, const QString &message);
00037     void openLogFile();
00038
00039     QFile logFile;
00040     QString m_logPath;
00041     QTextStream out;
00042 };
00043
00044 #endif // LOGS_H

```

6.4. settings.h

```

00001 #ifndef SETTINGS_H
00002 #define SETTINGS_H
00003
00004 #include <QObject>
00005 #include <QString>
00006 #include <QDir>
00007 #include "logs.h"
00008 #include "exceptions.h"
00009
00016 class Settings : public QObject
00017 {
00018     Q_OBJECT
00019     Q_PROPERTY(QString dbHost READ dbHost WRITE setDbHost NOTIFY dbHostChanged)
00020     Q_PROPERTY(int dbPort READ dbPort WRITE setDbPort NOTIFY dbPortChanged)
00021     Q_PROPERTY(QString dbName READ dbName WRITE setDbName NOTIFY dbNameChanged)
00022     Q_PROPERTY(QString dbUser READ dbUser WRITE setDbUser NOTIFY dbUserChanged)
00023     Q_PROPERTY(QString dbPassword READ dbPassword WRITE setDbPassword NOTIFY dbPasswordChanged)
00024     Q_PROPERTY(bool isServerMode READ isServerMode WRITE setIsServerMode NOTIFY isServerModeChanged)
00025     Q_PROPERTY(QString logPath READ logPath WRITE setLogPath NOTIFY logPathChanged)
00026     Q_PROPERTY(QString serverHost READ serverHost WRITE setServerHost NOTIFY serverHostChanged)
00027     Q_PROPERTY(int serverPort READ serverPort WRITE setServerPort NOTIFY serverPortChanged)
00028     Q_PROPERTY(QString reportsPath READ reportsPath WRITE setReportsPath NOTIFY reportsPathChanged)
00029
00030 public:
00036     explicit Settings(Logs *logger, QObject *parent = nullptr);
00037
00038     // Геттеры и сеттеры с краткими пояснениями
00039     QString dbHost() const { return m_dbHost; }
00040     void setDbHost(const QString &host);
00041
00042     int dbPort() const { return m_dbPort; }
00043     void setDbPort(int port);
00044
00045     QString dbName() const { return m_dbName; }
00046     void setDbName(const QString &name);
00047
00048     QString dbUser() const { return m_dbUser; }
00049     void setDbUser(const QString &user);
00050
00051     QString dbPassword() const { return m_dbPassword; }
00052     void setDbPassword(const QString &password);
00053
00054     bool isServerMode() const { return m_isServerMode; }
00055     void setIsServerMode(bool mode);
00056
00057     QString logPath() const { return m_logPath; }
00058     void setLogPath(const QString &path);
00059
00060     QString serverHost() const { return m_serverHost; }
00061     void setServerHost(const QString &host);
00062
00063     int serverPort() const { return m_serverPort; }
00064     void setServerPort(int port);
00065
00066     QString reportsPath() const { return m_reportsPath; }
00067     void setReportsPath(const QString &path);

```

```

00068
00070     Q_INVOKABLE void load();
00072     Q_INVOKABLE void save();
00073
00074 signals:
00075     void dbHostChanged();
00076     void dbPortChanged();
00077     void dbNameChanged();
00078     void dbUserChanged();
00079     void dbPasswordChanged();
00080     void isServerModeChanged();
00081     void logPathChanged();
00082     void serverHostChanged();
00083     void serverPortChanged();
00084     void reportsPathChanged();
00085
00086 private:
00087     QString m_dbHost;
00088     int m_dbPort;
00089     QString m_dbName;
00090     QString m_dbUser;
00091     QString m_dbPassword;
00092     bool m_isServerMode;
00093     QString m_logPath;
00094     Logs *logger;
00095     QString m_serverHost;
00096     int m_serverPort;
00097     QString m_reportsPath;
00098 };
00099
00100 #endif // SETTINGS_H

```

6.5. tcpclient.h

```

00001 #ifndef TCPCLIENT_H
00002 #define TCPCLIENT_H
00003
00004 #include <QObject>
00005 #include <QTcpSocket>
00006 #include <QVariantList>
00007 #include <QVariantMap>
00008 #include "logs.h"
00009 #include "exceptions.h"
00010
00011 class TcpClient : public QObject
00012 {
00013     Q_OBJECT
00014 public:
00015     explicit TcpClient(Logs *logger, QObject *parent = nullptr);
00016     ~TcpClient();
00017
00018     bool connectToServer(const QString &host, quint16 port);
00019     void disconnectFromServer();
00020     bool isConnected() const;
00021
00022     Q_INVOKABLE bool checkUser(const QString &login, const QString &password);
00023
00024     Q_INVOKABLE QString getUsername() const;
00025     Q_INVOKABLE QString getUserRole() const;
00026     Q_INVOKABLE int getUserId() const;
00027
00028     Q_INVOKABLE QVariantList getFoods();
00029
00030     Q_INVOKABLE QVariantList getProducts();
00031
00032     Q_INVOKABLE bool updateProductQuantity(int productId, int newQuantity);
00033
00034     Q_INVOKABLE bool addProductQuantity(int productId, int delta);
00035
00036     Q_INVOKABLE int createOrder(int tableNumber, int userId, const QString &specialRequests, double
totalCost);
00037
00038     Q_INVOKABLE bool addOrderItem(int orderId, int foodId, int quantity, double price);
00039
00040     Q_INVOKABLE bool checkAndConsume(int foodId, int quantity);
00041
00042     Q_INVOKABLE bool returnIngredients(int foodId, int quantity);
00043
00044     Q_INVOKABLE QVariantList getLowStockWarning(int foodId, int quantity);
00045
00046     Q_INVOKABLE QVariantMap getReportRevenue(const QString &from, const QString &to, int dishId, int
tableNumber, int waiterId);
00047
00048     Q_INVOKABLE QVariantList getRevenueByDay(const QString &from, const QString &to, int dishId, int

```

```

        tableNumber, int waiterId);
00151
00158     Q_INVOKABLE QVariantList getFoodPopularity(const QString &from, const QString &to);
00159
00161     Q_INVOKABLE QVariantList getReportLowStock();
00162
00164     Q_INVOKABLE QVariantList getDishesList();
00165
00167     Q_INVOKABLE QVariantList getWaitersList();
00168
00170     Q_INVOKABLE QStringList getLocalIps();
00171
00178     Q_INVOKABLE bool pingServer(const QString &host, int port);
00179
00186     Q_INVOKABLE bool reconnect(const QString &host, quint16 port);
00187
00199     Q_INVOKABLE bool saveReportToCSV(const QString &reportType,
00200                                     const QString &from,
00201                                     const QString &to,
00202                                     int dishId,
00203                                     int tableNumber,
00204                                     int waiterId,
00205                                     const QString &fileName);
00206
00207 signals:
00209     void errorOccurred(const QString &message);
00211     void dataChanged();
00213     void responseReceived(int requestId, bool success, const QVariantMap &data);
00214
00215 private:
00222     int sendRequest(const QString &command, const QVariantMap &data = QVariantMap());
00223
00230     bool waitForResponse(int requestId, QVariantMap &resultData);
00231
00232     QTcpSocket *socket;
00233     QByteArray buffer;
00234     int nextRequestId;
00235     QMap<int, QVariantMap> pendingResponses;
00236
00237     QString userName;
00238     QString userRole;
00239     int userId;
00240     Logs *logger;
00241 };
00242
00243 #endif // TCPCLIENT_H

```

6.6. tcpserver.h

```

00001 #ifndef TCPSERVER_H
00002 #define TCPSERVER_H
00003
00004 #include <QObject>
00005 #include <QTcpServer>
00006 #include <QTcpSocket>
00007 #include <QMap>
00008 #include "logs.h"
00009 #include "exceptions.h"
00010
00011 class DBWorker;
00012
00013 class ClientConnection : public QObject
00014 {
00015     Q_OBJECT
00016 public:
00017     ClientConnection(Logs *logger, qintptr socketDescriptor, DBWorker *dbWorker, QObject *parent =
00018                     nullptr);
00019     ~ClientConnection();
00020
00021 private slots:
00022     void onReadyRead();
00023     void onDisconnected();
00024
00025 private:
00026     void sendResponse(int requestId, bool success, const QVariantMap &data = QVariantMap());
00027
00028     QTcpSocket *socket;
00029     DBWorker *dbWorker;
00030     QByteArray buffer;
00031     Logs *logger;
00032 };
00033
00034 class TcpServer : public QObject
00035 {

```

```
00059     Q_OBJECT
00060 public:
00061     explicit TcpServer(Logs *logger, DBWorker *dbWorker, QObject *parent = nullptr);
00062     ~TcpServer();
00063
00064     bool start(quint16 port);
00065     void stop();
00066
00067 private slots:
00068     void onNewConnection();
00069
00070 private:
00071     QTcpServer *server;
00072     DBWorker *dbWorker;
00073     QList<ClientConnection*> connections;
00074     Logs *logger;
00075 };
00076
00077 #endif // TCPSERVER_H
```